Week 2 – Classification: Self-Study Tasks

Saurav · Tools: Python, pandas, scikit-learn

Note: the Week 2 dataset wasn't in the course repository, so for the practical part I used a synthetic customer-churn dataset (1,500 rows) that I generated myself.

Tuesday

1. Check Your Understanding

1. Is predicting the exact grade (0–100) classification?

No, it is regression because the target is a number. It becomes classification if the grade is turned into categories, e.g. pass/fail (grade ≥ 50) or grade bands.

2. 96% accuracy, but 96% of examples are one class. What should you check?

A model that always predicts the majority class also gets 96%. I would compare with a DummyClassifier baseline, look at the confusion matrix, and check precision and recall for the smaller class.

3. Why fit imputation after the train/test split?

The imputer learns values (e.g. the median) from the data. If it is fitted on all the data, the test set influences training (leakage) and the test score becomes too optimistic.

4. High precision but low recall — what error is common?

False negatives. When the model predicts positive it is usually right, but it misses many actual positive cases.

5. Why is a variable recorded after the outcome a problem?

It is data leakage. The variable is not available when the prediction is made and it indirectly reveals the answer. The model looks very good in testing but fails in real use.

2. scikit-learn workflow (VanderPlas)

Data is arranged as a feature matrix **X** (rows = observations, columns = features) and a target array **y**. Every model follows the same estimator API: choose the model class, set its hyperparameters, **fit()** it on training data, then **predict()** new data. In the Iris example, 4 flower measurements are used to predict one of 3 species.

3. Classification methods (Zaki & Meira)

Method	Idea	Assumption / note
Bayes classifier	Picks the class with the highest probability P(class | x)	Needs the full distribution of each class
Naive Bayes	Same idea, but features are treated separately	Features independent within each class; simple and fast
KNN	Majority class of the K nearest training points	Needs scaled features; K must be chosen
Decision tree	If-then splits on feature values	Easy to explain; overfits if too deep

4. Short practical check (Iris)

```
X, y = load_iris(return_X_y=True, as_frame=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)
knn = KNeighborsClassifier(n_neighbors=5).fit(X_train, y_train)
y_pred = knn.predict(X_test)
```

X has 150 rows and 4 features, and y has 3 balanced classes (50 each). The split gives 112 training and 38 test rows. KNN was fitted and produced predictions.

Wednesday – First classification model

Part A – Understand the problem

- **Observation:** one customer. **Target:** `churned` (1 = left within 3 months). This is **binary** classification.
- **Useful features:** age, tenure, contract type, payment method, region, monthly charges, support calls, satisfaction, usage.
- **Not features:** `customer_id` (an identifier) and `exit_survey_completed` (recorded after the customer leaves, which is leakage).
- **Balance:** not balanced. 1001 stayed (67%) and 499 churned (33%).
- **Errors:** a false negative means a churner is missed (the customer is lost). A false positive means an unnecessary retention offer. Here false negatives are worse.

Part B – Prepare X and y

Missing values are in satisfaction_score (107) and avg_monthly_usage_gb (63). They are filled with the median. The categorical columns (contract, payment, region) are one-hot encoded. The numerical columns are scaled for logistic regression.

Part C – Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)
```

This gives 1125 training and 375 test rows, and both have 33% churn.

Part D – Baseline

`DummyClassifier(strategy='most_frequent')` always predicts "stayed". It gets accuracy **0.667** but recall 0, so it never finds a churner.

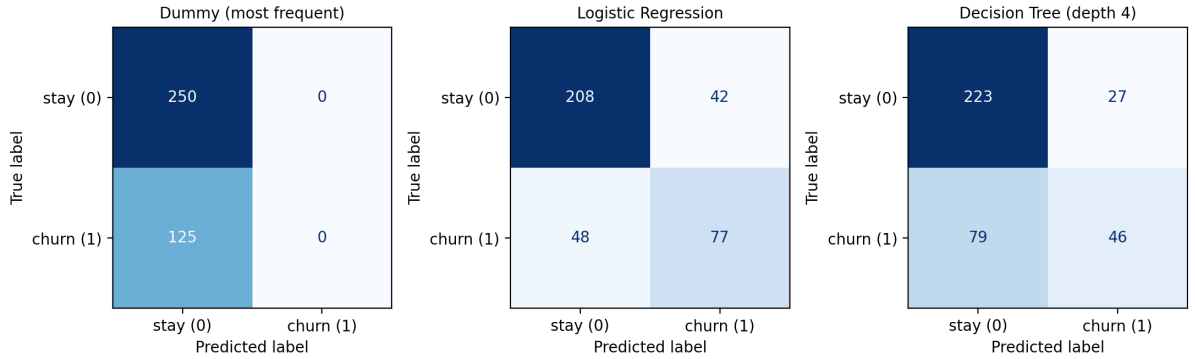
Part E – Classifiers

```
prep = ColumnTransformer([("num", Pipeline([("impute", SimpleImputer(strategy="median")),
                                            ("scale", StandardScaler())]), num),
                          ("cat", OneHotEncoder(handle_unknown="ignore"), cat)])
logreg = Pipeline([("prep", prep),
                   ("clf", LogisticRegression(max_iter=1000))])
tree = Pipeline([("prep", prep),
                 ("clf", DecisionTreeClassifier(max_depth=4, random_state=0))])
```

Classifier	Preprocessing	Test accuracy	Observation
Logistic Regression	impute, scale, one-hot	0.760	Clearly better than the baseline
Decision Tree (depth 4)	impute, one-hot	0.717	Better than the baseline but misses many churners

Thursday – Evaluation and leakage

1. Evaluation



Model	Baseline?	Accuracy	Precision	Recall	Main observation
Dummy (most frequent)	Yes	0.667	0.000	0.000	Never predicts churn
Logistic Regression	No	0.760	0.647	0.616	Finds 77 of 125 churners
Decision Tree	No	0.717	0.630	0.368	Misses 79 of 125 churners

Which model is more useful? Logistic Regression. Its accuracy is higher, but more importantly its recall is much higher (0.62 vs 0.37) with similar precision. Missing churners is the most important error here.

2. Data leakage and pipelines

All preprocessing is inside a Pipeline, so the imputer and scaler are fitted only on the training data and then applied to the test data.

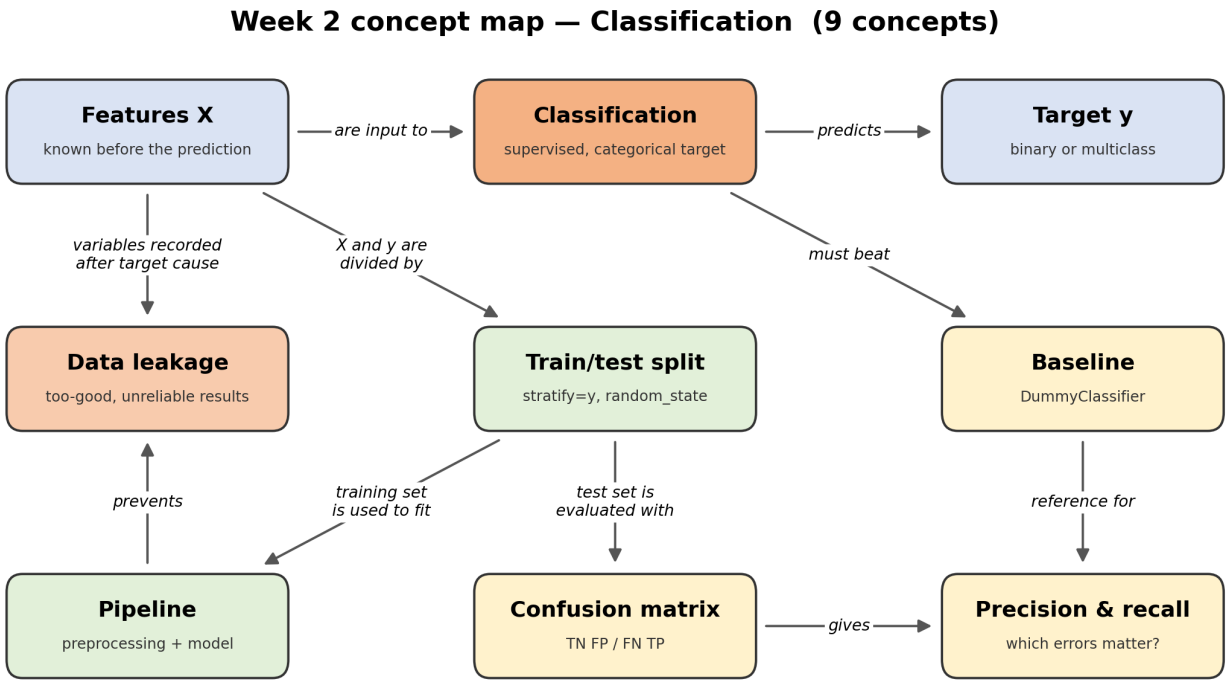
Leakage test: I added `exit_survey_completed`, a variable recorded after the customer leaves. Accuracy jumped from 0.76 to **0.96**. This result is unreliable because the survey only exists after churn has happened. At prediction time the model would not have it, so it would not work in practice.

3. Compare and interpret

- **Better than the baseline:** both classifiers. Logistic Regression is the clearest improvement (0.76 vs 0.67, recall 0.62 vs 0).
- **Mistakes:** Logistic Regression misses 48 churners (false negatives) and gives 42 false alarms (false positives).
- **Large enough data?** The test set has only 375 rows, so differences of 1–3% between models could be random. The gap between the models and the baseline is clear.
- **Investigate further:** lowering the decision threshold of Logistic Regression to catch more churners.
- **Limitation:** synthetic data, one train/test split and a small test set. The results may not hold on real data.

Friday

Concept map (9 concepts)



Student: Saurav · Arrows read as sentences, e.g. “Features X → are input to → Classification”

Practical verification values

Quantity	Value
Observations	1500
Class distribution	1001 stayed / 499 churned
Train / test size	1125 / 375
Baseline accuracy	0.667
Logistic Regression confusion matrix	TN 208, FP 42, FN 48, TP 77
Logistic Regression accuracy / precision / recall	0.760 / 0.647 / 0.616

The group Project Data Plan update is submitted separately by the group.